

Atividade - Lab 01

Integrantes

Gustavo Kiyoshi Ikeda - RA 10439179

Pedro Montarroyos de Pinho - RA 10440213

Felipe Marques Leite Martha - RA 10437877

Repositório do código-fonte: <https://github.com/pedrompinho2/Atividade-1>

2. Prints

```
● @pedrompinho2 →/workspaces/Atividade-1 (main) $ gcc -Wall -o atividade atividade_01_nome_ra.c
○ @pedrompinho2 →/workspaces/Atividade-1 (main) $ ./atividade
```

SISTEMA DE CADASTRO DE PRODUTOS

Menu:

1. Adicionar produto
2. Listar produtos
3. Buscar produto
4. Atualizar estoque
5. Remover produto
6. Sair

Opcao: █

Opcao: 1

--- Adicionar Produto ---

Nome: Caderno

Preco: 20

Quantidade: 4

Produto adicionado com codigo 1!

Opcao: 2

--- Lista de Produtos ---

Codigo	Nome	Preco	Qtd	Valor	Estoque
1	Caderno	20.00	4	80.00	
2	caneta	2.00	4	8.00	

Valor total do estoque: R\$ 88.00

Opcao: 3

--- Buscar Produto ---

Codigo do produto: 2

Nome: caneta

Preco: 2.00

Quantidade: 4

```
Opcao: 3

--- Buscar Produto ---
Codigo do produto: 4
Produto nao encontrado.
```

```
Opcao: 2

--- Lista de Produtos ---
-----
| Codigo | Nome           | Preco  | Qtd  | Valor Estoque |
-----
| 1      | Caderno        | 20.00  | 4    | 80.00          |
| 2      | caneta         | 2.00   | 3    | 6.00           |
-----
Valor total do estoque: R$ 86.00
```

```
Opcao: 5

Remover Produto
Codigo do produto: 1
Produto "Caderno" removido
```

```
Opcao: 2

--- Lista de Produtos ---
-----
| Codigo | Nome           | Preco  | Qtd  | Valor Estoque |
-----
| 2      | caneta         | 2.00   | 3    | 6.00           |
-----
Valor total do estoque: R$ 6.00
```

Prints - Valgrind

```
gustavoiked@DESKTOP-D830RBH:~/Docs/Estudo/dist1.c$ valgrind --leak-check=full ./dist1
==2962== Memcheck, a memory error detector
==2962== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==2962== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==2962== Command: ./dist1
==2962==

SISTEMA DE CADASTRO DE PRODUTOS
Menu:
1. Adicionar produto
2. Listar produtos
3. Buscar produto
4. Atualizar estoque
5. Remover produto
6. Sair
```

```
Opcao: 1

--- Adicionar Produto ---
Nome: Caderno
Preco: 30
Quantidade: 1
Produto adicionado com codigo 1!
```

```
Opcao: 2

--- Lista de Produtos ---
-----
|Codigo|Nome          |Preco  |Qtd  |Valor Estoque|
-----
| 1    |Caderno       |30.00  |1    |30.00        |
-----
Valor total do estoque: R$ 30.00
```

```
Opcao: 3

--- Buscar Produto ---
Codigo do produto: 1

Nome: Caderno
Preco: 30.00
Quantidade: 1
```

```
Opcao: 4

Atualizar Estoque
Codigo do produto: 1
Quantidade atual: 1
Nova quantidade: 2
Estoque atualizado
```

```
Opcao: 5

Remover Produto
Codigo do produto: 1
Produto "Caderno" removido
```

```

SISTEMA DE CADASTRO DE PRODUTOS
Menu:
1. Adicionar produto
2. Listar produtos
3. Buscar produto
4. Atualizar estoque
5. Remover produto
6. Sair

Opcao: 6

Liberando memoria
Vetor de produtos liberado
encerrado
==2962==
==2962== HEAP SUMMARY:
==2962==    in use at exit: 0 bytes in 0 blocks
==2962==   total heap usage: 4 allocs, 4 frees, 2,080 bytes allocated
==2962==
==2962== All heap blocks were freed -- no leaks are possible
==2962==
==2962== For lists of detected and suppressed errors, rerun with: -s
==2962== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

3. Perguntas

1. Por que usamos `char*` para o nome em vez de `char nome[50]`? Qual a vantagem e desvantagem de cada abordagem?

Com `char nome[50]` o espaço é fixo: sempre 50 bytes reservados, desperdiçando memória se o nome for curto e cortando o nome se for maior que o limite. Com `char*` e `malloc(strlen(nome)+1)`, alocamos exatamente o tamanho necessário para aquele nome, em tempo de execução. A vantagem do `char*` é o uso de memória sob medida, sem limite fixo; a desvantagem é que precisamos gerenciar a alocação e o free manualmente, sob risco de memory leak. O vetor fixo é mais simples e sem risco de leak, mas desperdiça memória ou estoura o tamanho.

2. Explique a diferença entre `Produto*` e `Produto**` no contexto do seu código. Em que situações você usaria cada um?

`Produto*` aponta para um produto ou para o primeiro elemento de um vetor de produtos e é usado quando a função só precisa ler ou alterar dados já existentes, sem mudar o endereço do vetor. Já `produto**` É um ponteiro para o próprio ponteiro estoque, necessário quando a função faz realloc e precisa que essa mudança de endereço volte para a variável original no main.

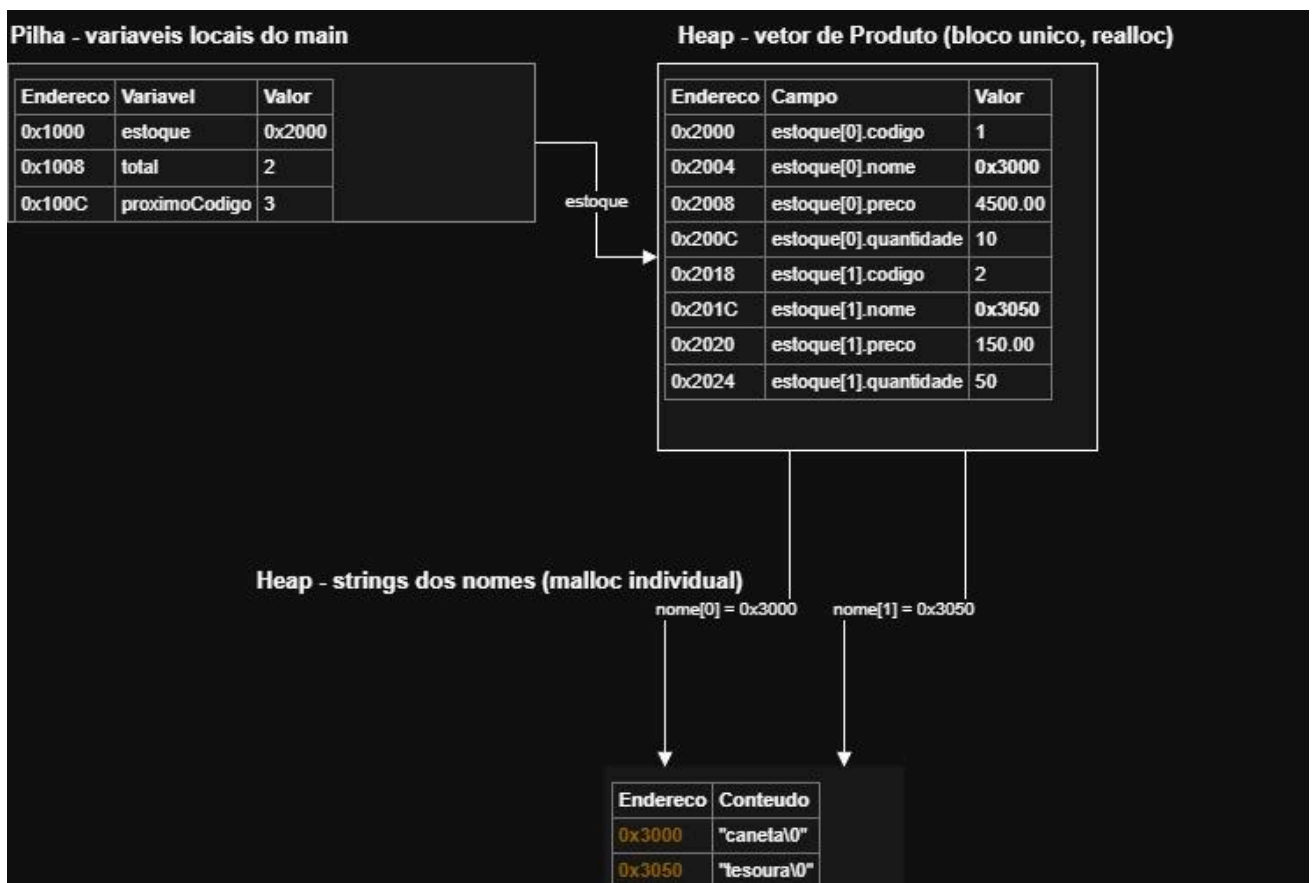
3. Na função de adicionar produto, por que precisamos de realloc? O que aconteceria se usássemos apenas malloc?

O realloc permite aumentar o vetor conforme a necessidade, preservando os dados já existentes. Usando só malloc, teríamos que fixar um tamanho máximo no início ou recriar um vetor novo a cada produto, copiando todos os itens antigos e liberando o vetor anterior.

4. Na função de remover produto, por que a ordem de liberação de memória importa? O que aconteceria se liberássemos o vetor de produtos antes de liberar os nomes?

Na função `remover_produto` a ordem de liberação de memória importa, pois caso o nome do produto fosse liberado depois do vetor, os endereços dos nomes seriam perdidos, tornando-se impossível acessá-los, desta forma causando um memory leak.

5. Desenhe o diagrama de memória mostrando como os produtos estão organizados após adicionar 3 produtos.



6. O que é um memory leak? Aponte no seu código onde isso poderia ocorrer se você não tivesse cuidado.

Memory leak é uma memória alocada que nunca é liberada antes de se perder a última referência a ela, ficando inacessível até o fim do programa, fazendo ele exigir cada vez mais memória. No código, isso poderia ocorrer: na função `remover_produto`, caso o nome fosse liberado depois do deslocamento/realloc do vetor

7. Por que a função de busca retorna um ponteiro (`Produto*`) em vez de retornar

o produto por valor (Produto)?

Se retornasse Produto por valor, o C copiaria a struct inteira — teríamos acesso aos dados, mas alterações nessa cópia não afetariam o produto original no vetor. Isso quebraria `atualizar_estoque`, que depende de alterar a quantidade diretamente no vetor original a partir do resultado da busca. Retornando `Produto*`, recebemos o endereço real do produto dentro do vetor, então qualquer alteração feita através dele modifica o dado de fato. Além disso, retornar `NULL` é a forma natural de sinalizar 'não encontrado'.

A função busca retorna um ponteiro (`Produto*`) ao invés do produto por valor, pois caso ele retornasse o produto, as mudanças feitas nele não afetariam o produto real, apenas a cópia, desta forma impactando diretamente na função `atualizar_estoque`.

8. Na atualização de estoque, por que usar passagem por referência? O que aconteceria se você passasse o produto por valor?

A função `atualizar_estoque` deve usar passagem por referência, pois apenas dessa forma é possível alterar o dado onde ele realmente está armazenado. Se o produto fosse passado por valor, a função receberia uma cópia e qualquer alteração nessa cópia seria perdida ao final da função, e o produto original no vetor estoque continuaria com os dados antigos.